



IN PYTHON

Conditional Statements & Loops

DEFINITION

A conditional statement is a program construct that lets you make a decision about which part of the code should run.

The Five Forms

1. if-else
2. if-else (nested)
3. if-elif-else ladder
4. match-case (Python 3.10+)
5. ternary (conditional expression)

1. if-else

Python has no braces and no semicolons — indentation itself marks which lines belong to the if block. A colon (:) always follows the condition.

PYTHON

```
age = 31
if age >= 18:
    print("He can vote")
else:
    print("He cannot")
```

★NOTE

- else cannot exist without an if.
- if can exist on its own, without an else.
- Indentation is not a style choice here — it's how Python knows a line belongs to the if/else block.

PYTHON

```
age = 30
if age < 50:
    print("You are young")
```

ROUGH WORK

Comparison Operators

After if, there should be a condition — an expression that evaluates to True or False. Python needs no parentheses around it (though you may add them for clarity).

PYTHON

```
10 > 3          # True
3 == 4          # False
"ah" == "hi"    # False
```

Comparison Operators

Operator	Meaning
>=	More than or equal
<=	Less than or equal
==	Equal to
!=	Not equal to
<	Less than
>	Greater than

3. if-elif-else ladder

PYTHON

```
if C1:
    pass # some code
elif C2:
    pass # code
elif C3:
    pass # code
else:
    pass # code
```

★NOTE

- An if-elif ladder can be complete without a final else.
- Best practice: add a default/final else to handle edge cases, or random/wrong inputs, in real-world scenarios.
- elif is Python's single keyword for 'else if' — there is no separate else-if pair.

Nested if

PYTHON

```
age = 20
has_licence = True

if age >= 18:
    if has_licence:
        print("can drive")
    else:
        print("cannot drive")
else:
    print("Too young")
```

How the ladder actually evaluates

★NOTE

If an if fails, the elif's below it are checked next, in sequential order — and the final else runs only if every condition above it has failed.

- Every if in a plain, independent block checks its own condition regardless of what came before.
- An elif only gets checked when the condition above it in the same ladder has failed.

Compare two versions of the same idea — separate, independent if statements versus a proper if-elif ladder:

PYTHON

```
# Version A: independent ifs (NOT a ladder)
a = 10
if a > 3: print("A")
if a > 2: print("B")
if a > 2: print("E")
# every condition is checked, so all three can fire
```

Output: ABE

PYTHON

```
# Version B: if-elif ladder
a = 10
if a > 3:
    print("A")
elif a > 2:
    print("B")
else:
    print("E")
# first true branch wins, the rest are skipped
```

Output: A

Logical Operators

Python spells its logical operators out as words instead of symbols — no `&&`, `||`, or `!`.

Logical Operators

Keyword	Meaning
and	AND
or	OR
not	NOT

and — true only when both sides are true:

AND Truth Table

A	B	A and B
True	True	True
True	False	False
False	True	False
False	False	False

or — true when at least one side is true:

OR Truth Table

A	B	A or B
False	False	False
False	True	True
True	False	True
True	True	True

4. match-case (Python 3.10+)

Python has no switch statement. Since version 3.10 it has match-case, which plays a similar role — matching one expression against several patterns.

PYTHON

```
match expr:
    case v1:
        pass # statement1
    case v2:
        pass # statement2
```

```
case _:
    pass # default
```

★NOTE

- **No break needed** — unlike C++'s switch, Python does not fall through to the next case.
- **case _** is the wildcard/default — it matches anything not already caught above it.
- **expr** can be matched against values, types, or even structural patterns (tuples, lists, objects).
- **On Python versions before 3.10**, an if-elif ladder is the standard stand-in for switch.

Operator Precedence (High → Low)

Operator Precedence

Level	Operators
1 (highest)	()
2	**
3	unary +, -, ~
4	* // %
5	+ -
6	== != < > <= >=
7	not
8	and
9	or
10 (lowest)	ternary / =

5. Ternary (Conditional Expression)

PYTHON

```
value_if_true if condition else value_if_false
```

A compact, inline stand-in for a simple if-else that returns a value. Python reads it almost like an English sentence, in the opposite order to C++'s condition ? a : b.

PYTHON

```
age = 20
```

```
status = "Adult" if age >= 18 else "Minor"
print(status)
```

Output: Adult

Loops

DEFINITION

A loop repeats a block of code until a stated condition becomes false, so you don't have to write the same statements out by hand.

1. for loop

Python's for loop is built to iterate over a sequence directly — a range of numbers, a list, a string — rather than manually tracking a counter.

PYTHON

```
for i in range(5):
    print(i, end=" ")
```

Output: 0 1 2 3 4

range(5) produces the sequence 0, 1, 2, 3, 4. On each pass, i takes the next value from that sequence — once the sequence is exhausted, the loop ends on its own; there's no separate condition to check.

PYTHON

```
fruits = ["apple", "banana", "cherry"]
for f in fruits:
    print(f)
```

Reads naturally as "for each f in fruits." f takes each element's value in turn, in order — no index variable to manage.

2. while loop

A while loop only has a condition — you set up and update the counter yourself, outside the loop line.

PYTHON

```
i = 0
while i < 5:
    print(i, end=" ")
    i += 1
```

Output: 0 1 2 3 4

The condition is checked before every single pass, including the very first one. If $i < 5$ were false to begin with, the body would not run even once.

★NOTE

Python has no do-while. If you need the body to run at least once, use while True with a break at the end of the block:

PYTHON

```
while True:
    print("runs at least once")
    break
```

3. break and continue

break exits the loop immediately, skipping every remaining iteration:

PYTHON

```
for i in range(10):
    if i == 5:
        break
    print(i, end=" ")
```

Output: 0 1 2 3 4

Once i becomes 5, break fires and the loop ends right there — 5, 6, 7, 8, 9 are never printed.

continue skips only the current pass and jumps straight to the next value in the sequence:

PYTHON

```
for i in range(5):
    if i == 2:
        continue
    print(i, end=" ")
```

Output: 0 1 3 4

When i is 2, continue skips the print for that pass only — the loop itself keeps going, so 3 and 4 still print.

4. Nested loops

PYTHON

```
for i in range(1, 3):
    for j in range(1, 4):
        print(f"{i}{j}", end=" ")
```

Output: 11 12 13 21 22 23

The inner loop runs all the way through ($j = 1, 2, 3$) for every single pass of the outer loop, before the outer loop moves on to its next value of i .

5. The loop else clause — a Python-only feature

Both for and while loops in Python can carry an else block. This has nothing to do with if/else — it runs once the loop finishes normally, on its own terms.

PYTHON

```
for i in range(5):
    print(i)
else:
    print("Loop finished normally")
```

★NOTE

else on a loop runs only if the loop completes without hitting a break. If break fires, else is skipped entirely — this is the one thing worth memorizing about it.

- **No C++ equivalent** — this construct is unique to Python's for and while.
- **A common use: searching a list, then using else to report "not found" only if the loop never broke out early.**

Compare the same loop with a break added — the else block is skipped:

PYTHON

```
for i in range(5):
    if i == 3:
        break
    print(i)
else:
    print("This will NOT print")
```

Output: 0 1 2 (else is skipped, because break fired)

Infinite loops

while True: has no condition that ever becomes false on its own — it's Python's standard way to write "loop forever, until something inside decides to stop it."

PYTHON

```
while True:
    answer = input("Type 'quit' to stop: ")
    if answer == "quit":
        break
```

★NOTE

- Watch for accidental infinite loops — forgetting to update the loop variable inside a while, or a condition that never becomes false.
- for loops over a fixed range or sequence can't run forever by accident — that risk is really a while-loop concern.

End of Notes — Conditional Statements & Loops[NextGen Data Minds](#)